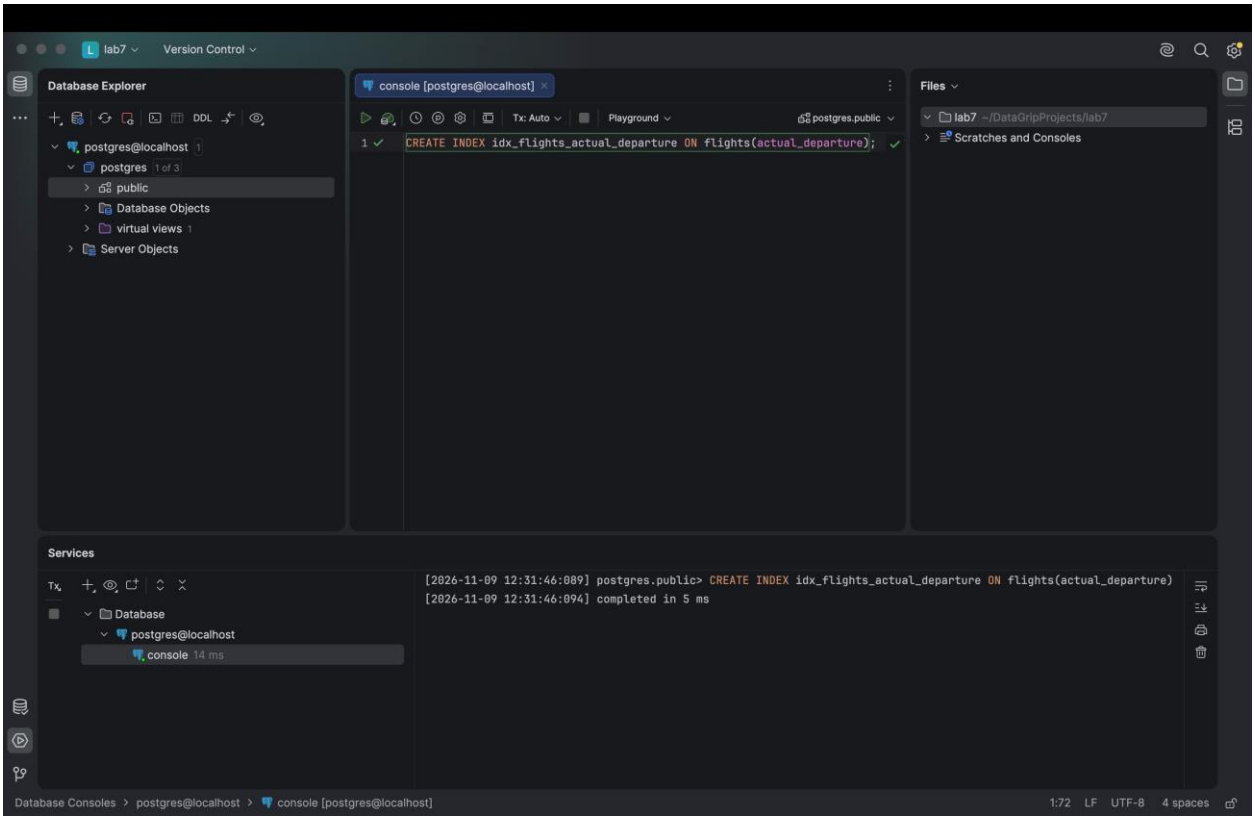
Laboratory work 7

We continue to work with the database from previous laboratory work. Take a full-page screenshot that covers the code and results of each task.

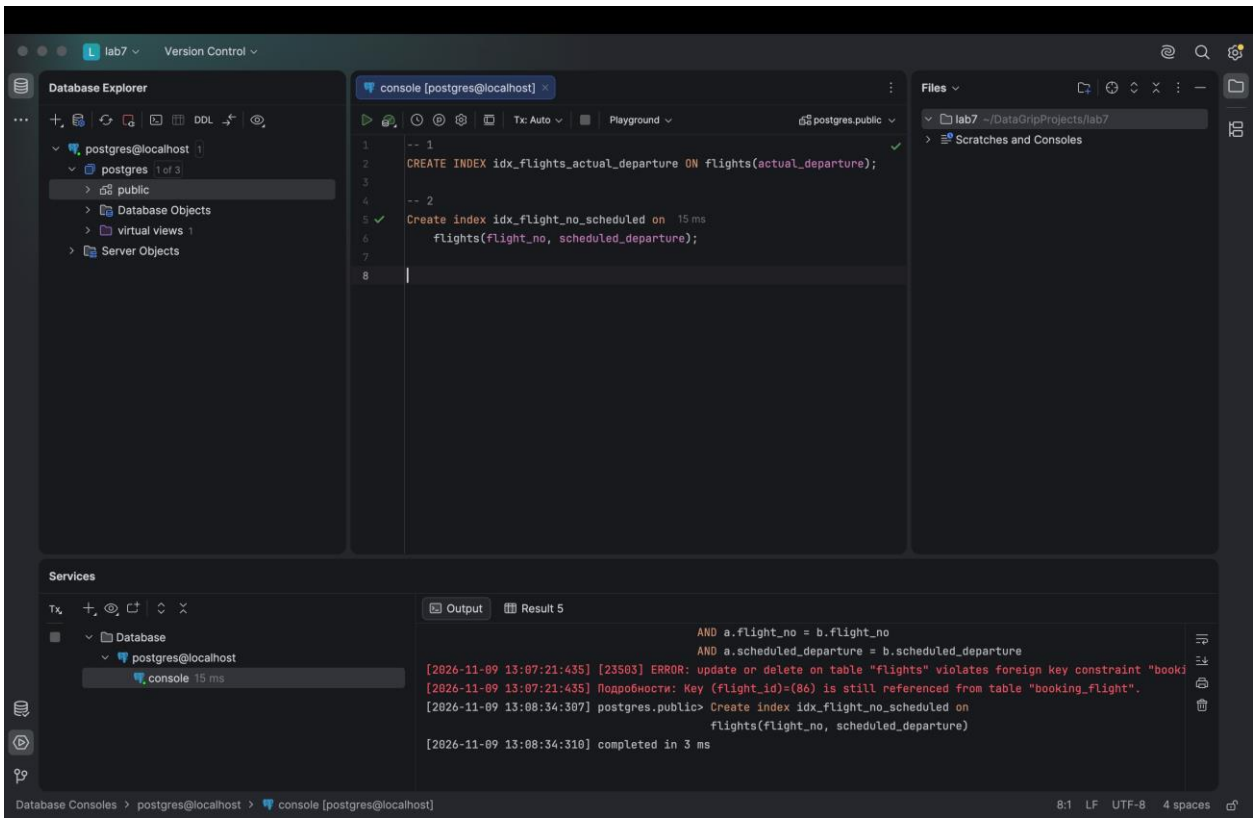
Tasks:

INDEX

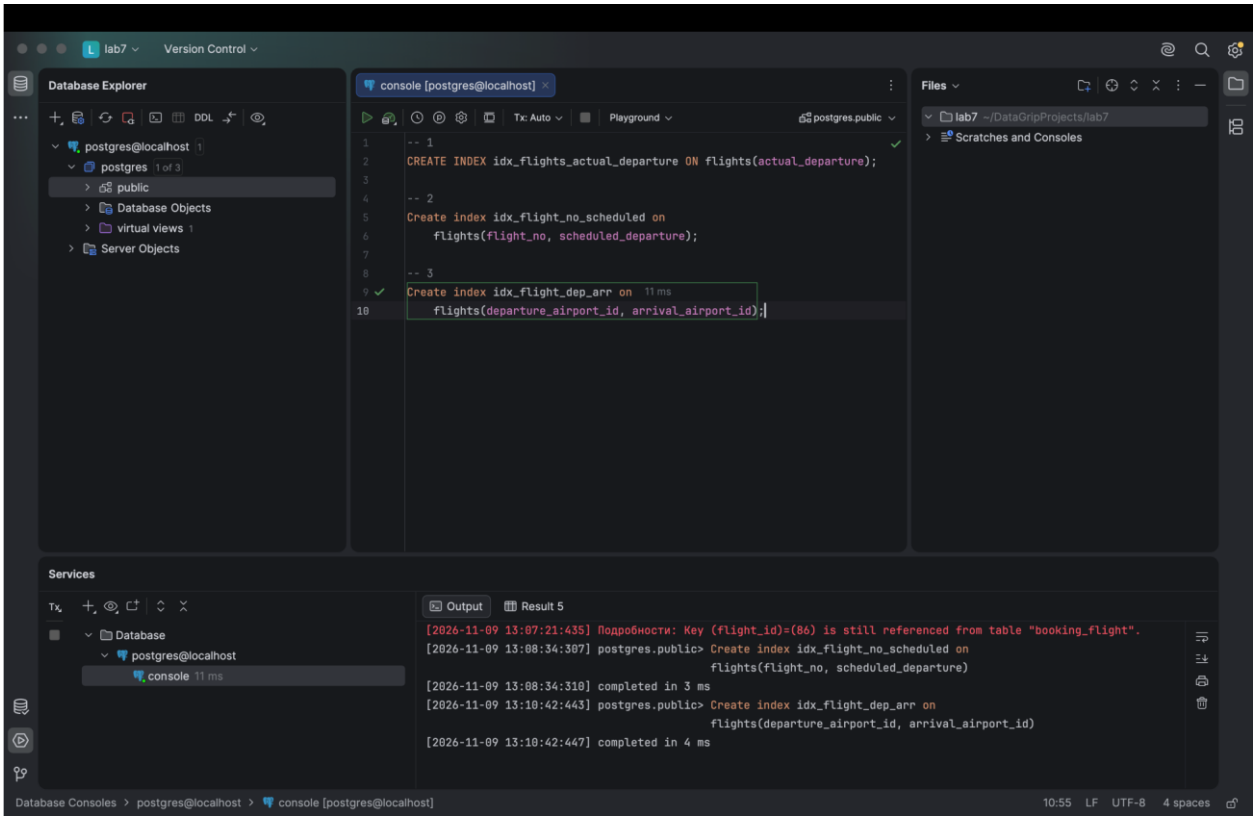
1. Create an index on the actual_departure column in the flights table.



2. Create a unique index to ensure flight_no and scheduled_departure combinations are unique.



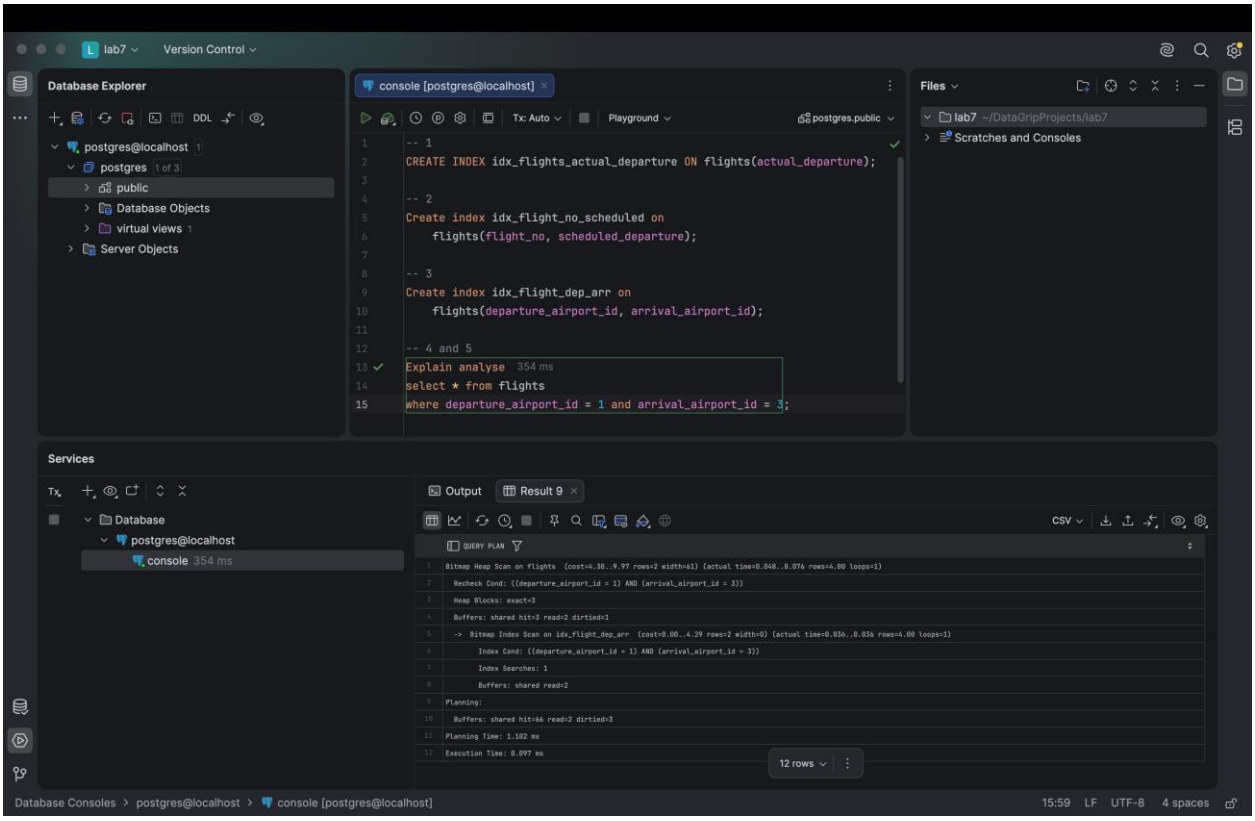
3. Create a composite index on the departure_airport_id and arrival_airport_id columns.



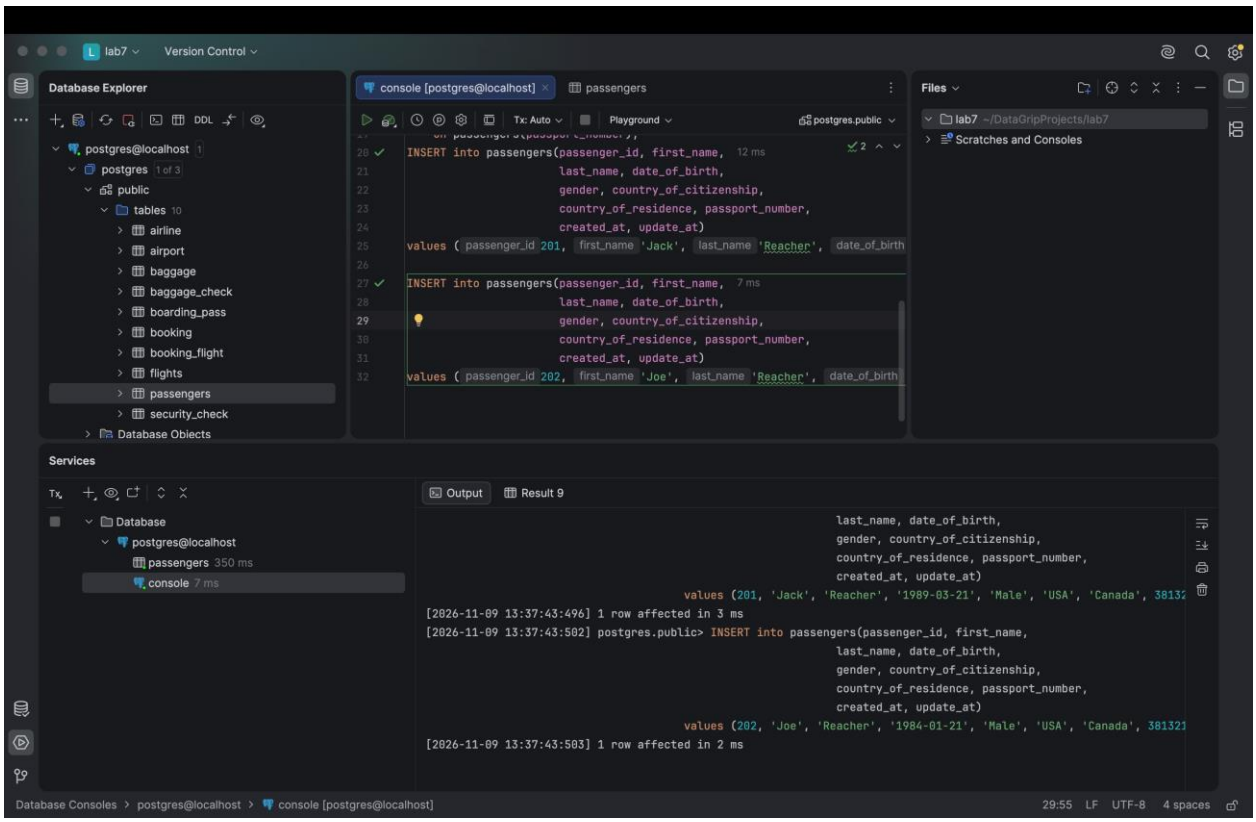
4. Evaluate the difference in query performance with and without indexes. Measure performance differences.

Without an index, PostgreSQL performs a sequential scan (a sequential traversal of all rows in the table). If a suitable index is available, the planner chooses an index scan or a bitmap index scan, which significantly reduces the number of data blocks read and the execution time.

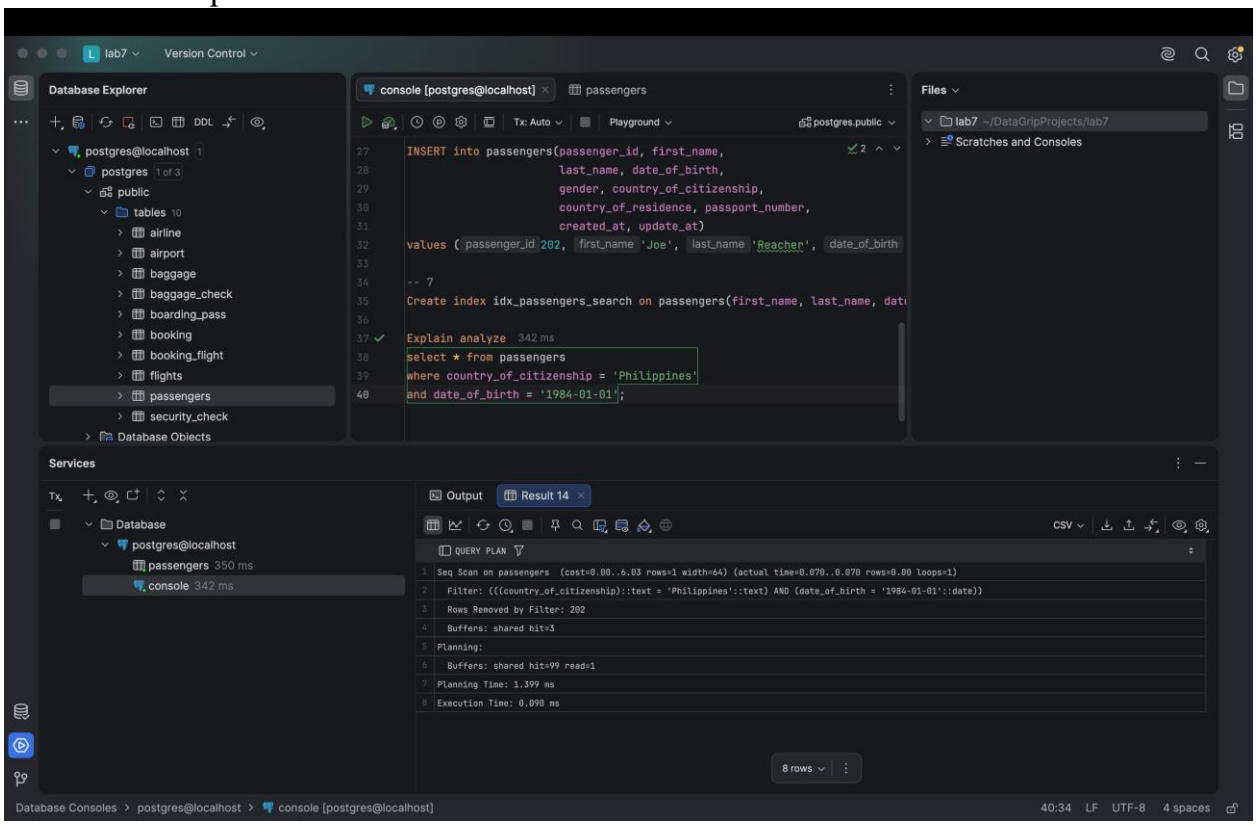
5. Use EXPLAIN ANALYZE to check index usage in a query filtering by departure_airport and arrival_airport.



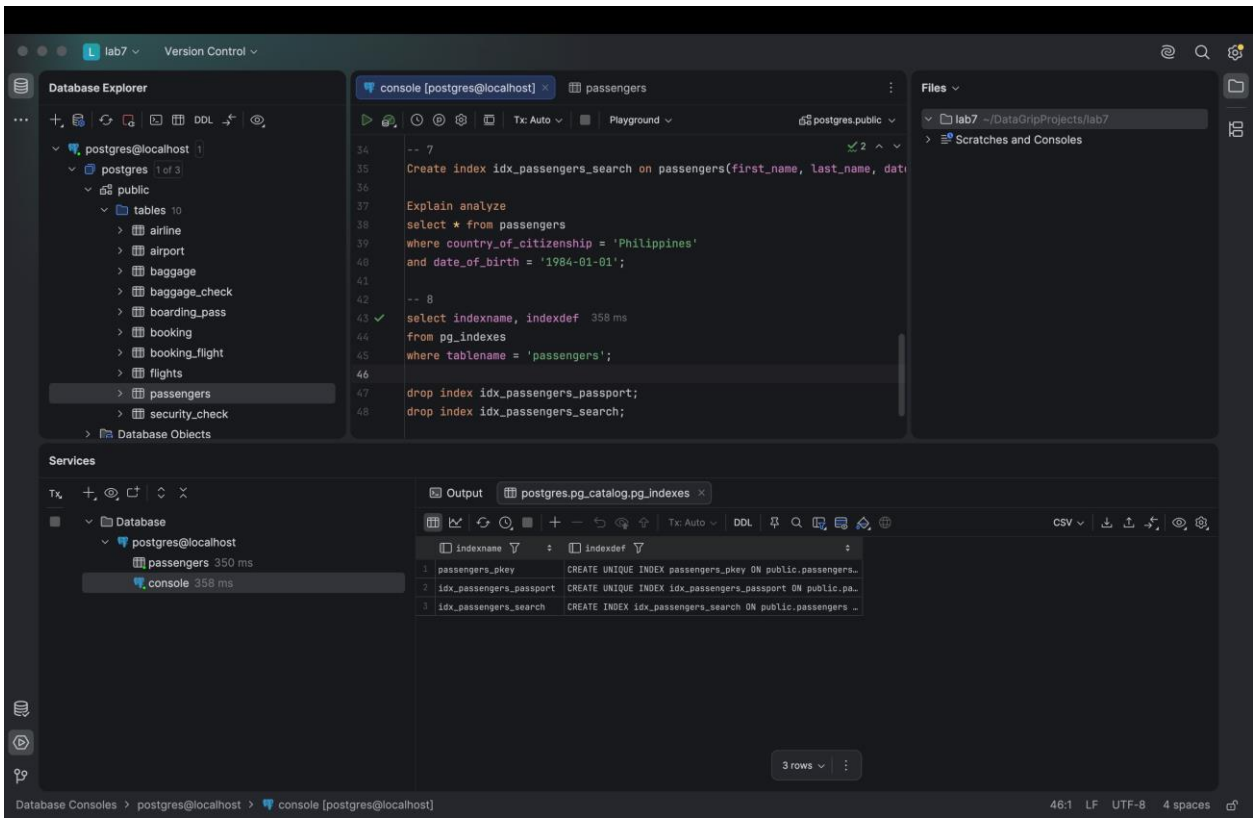
6. Create a unique index for the passport_number of the Passengers table. Check if the index was created or not. Insert into the table two new passengers.
   Explain in your own words what is going on in the output?



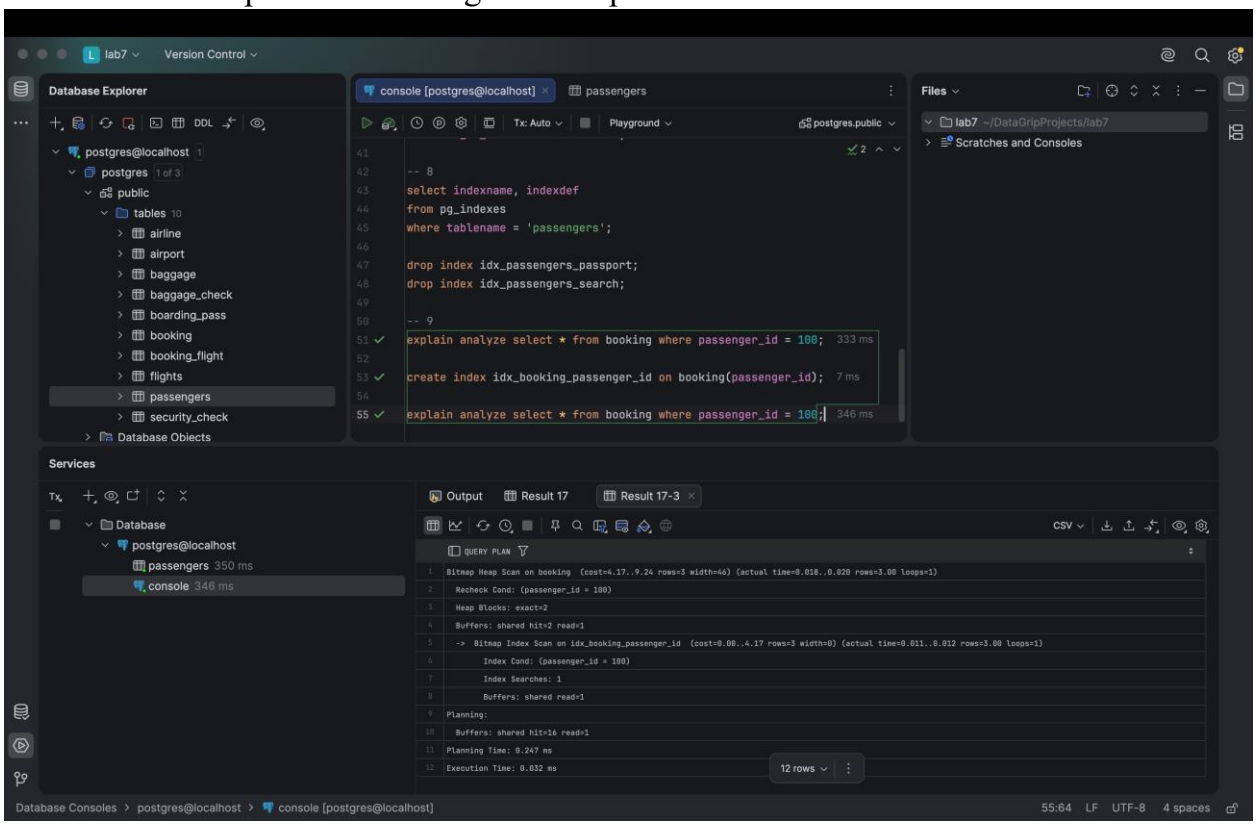
7. Create an index for the Passengers table. Use for that first name, last name, date of birth and country of citizenship. Then, write a SQL query to find a passenger who was born in Philippines and was born in 1984 and check if the query uses indexes or not. Give an explanation of the results.



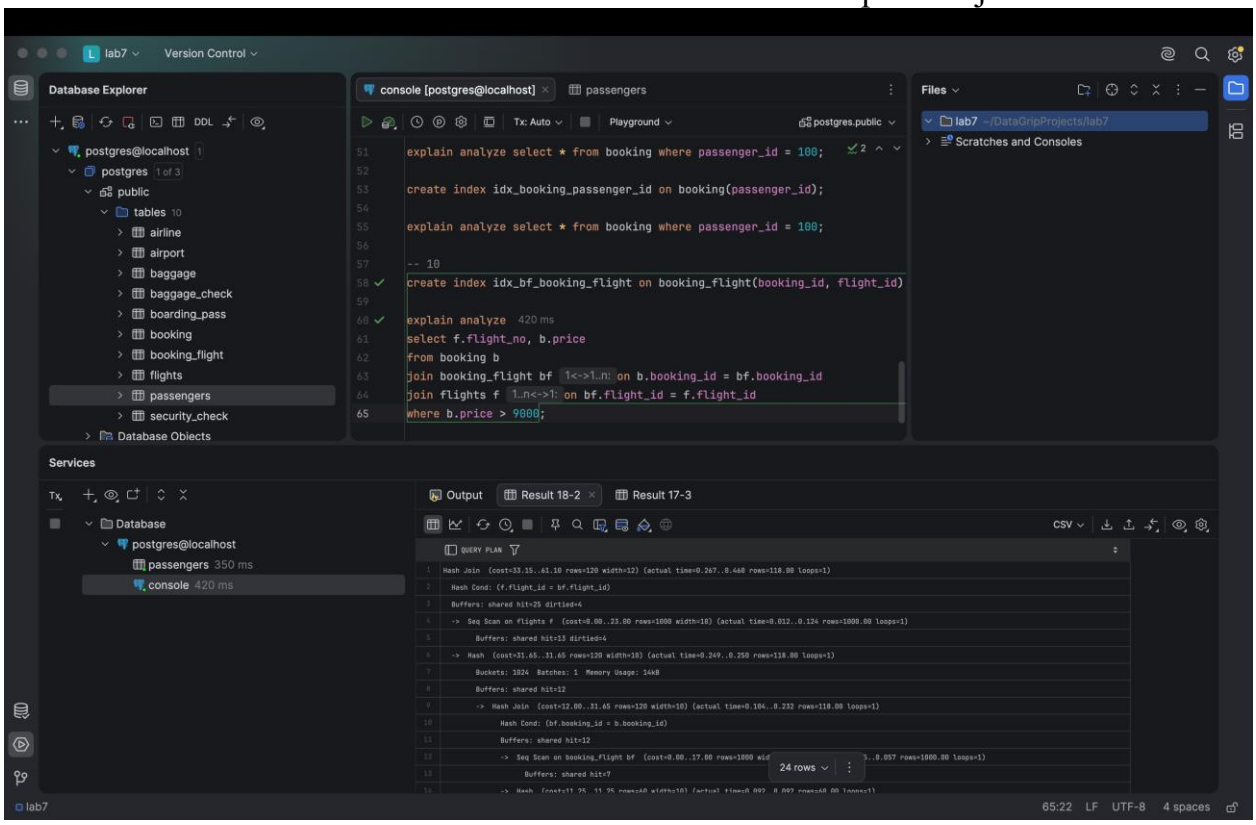
8. Write a SQL query to list indexes for table Passengers. After, delete the created indexes.



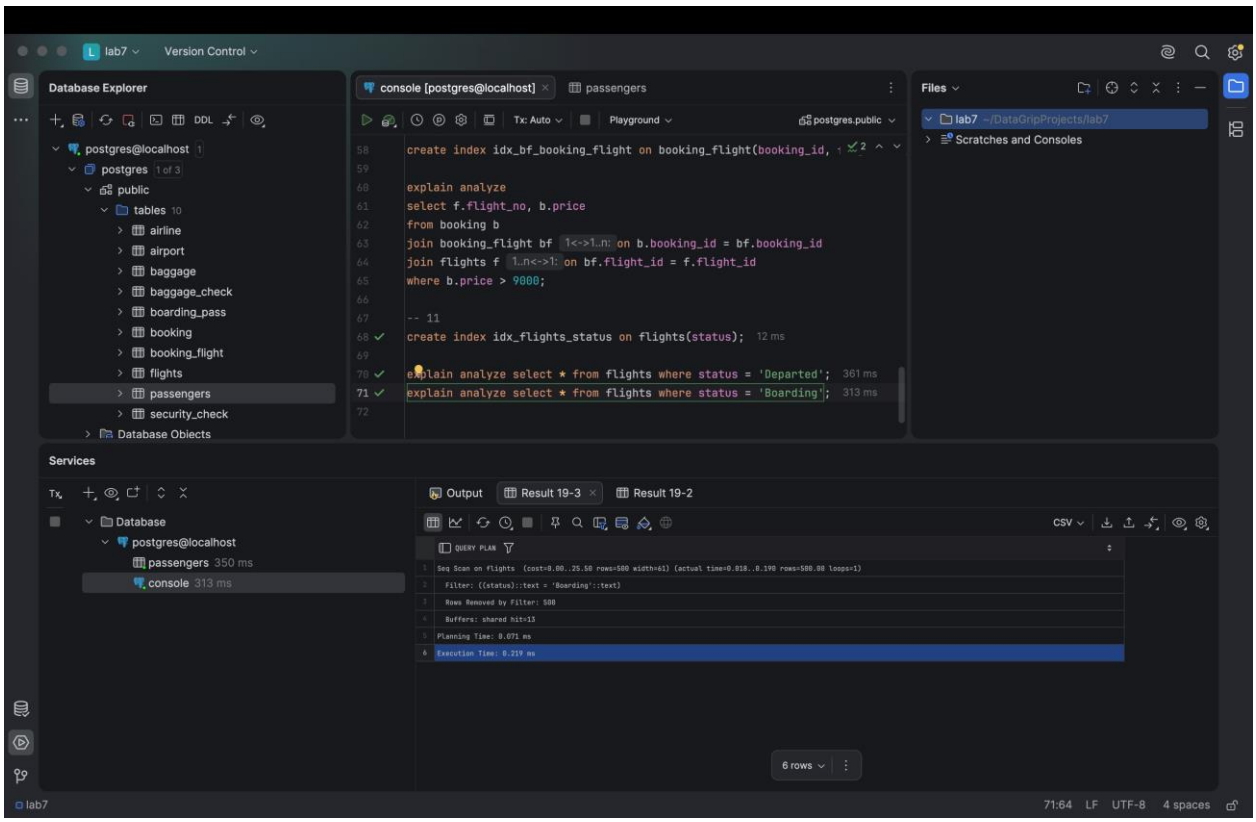
9. Create an index on the passenger_id column of the booking table. Using EXPLAIN ANALYZE, compare the execution plan and execution time of a query that selects all bookings of one passenger (for example, passenger_id = 100) before and after creating the index. Explain what changed in the plan.



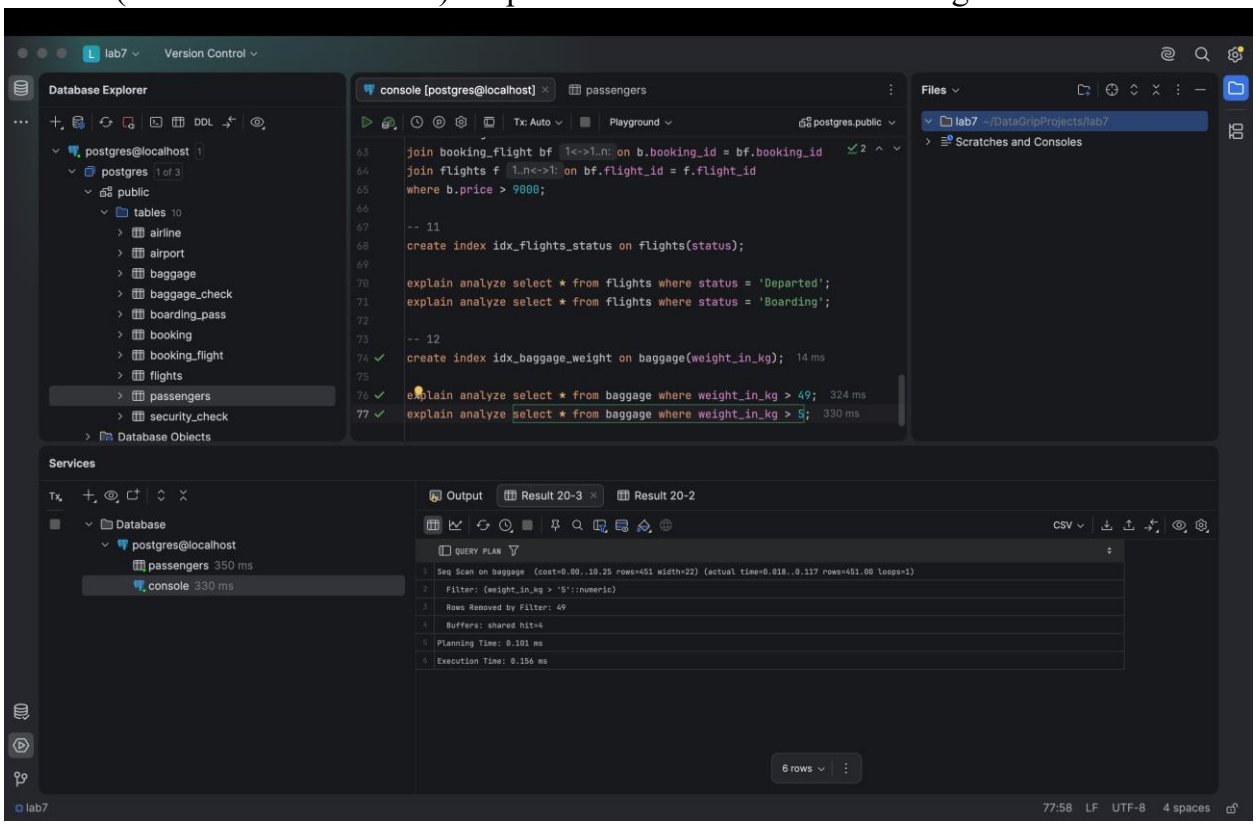
10. Create a composite index on the booking_flight table using the booking_id and flight_id columns. Write a query that joins booking, booking_flight and flights and returns the flight_no and price for bookings with a price greater than 9000. Check with EXPLAIN ANALYZE which indexes are used in each step of the join.



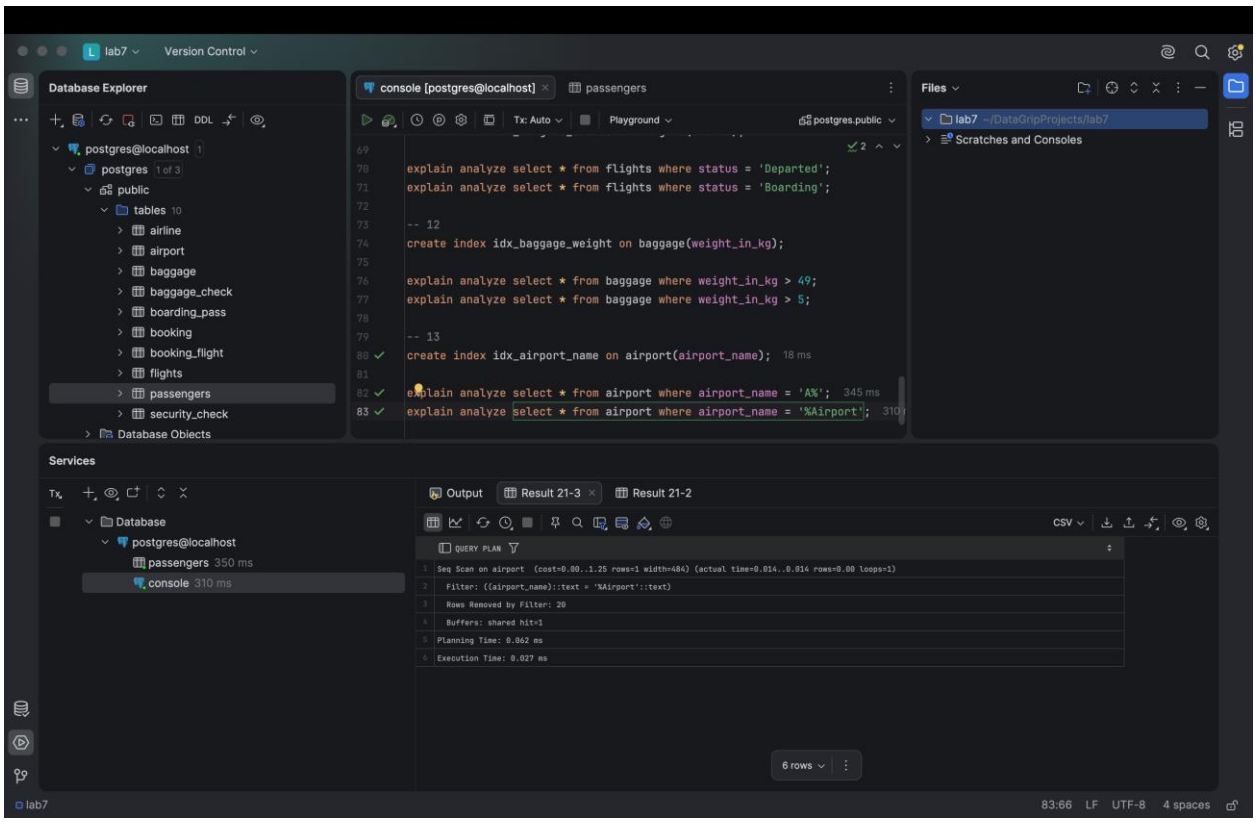
11. Create an index on the status column of the flights table. Run EXPLAIN ANALYZE for the query with status = 'Departed' and for the query with status = 'Boarding'. Compare the plans and explain, in your own words, why the optimizer may ignore the index for one of the values (hint: selectivity).



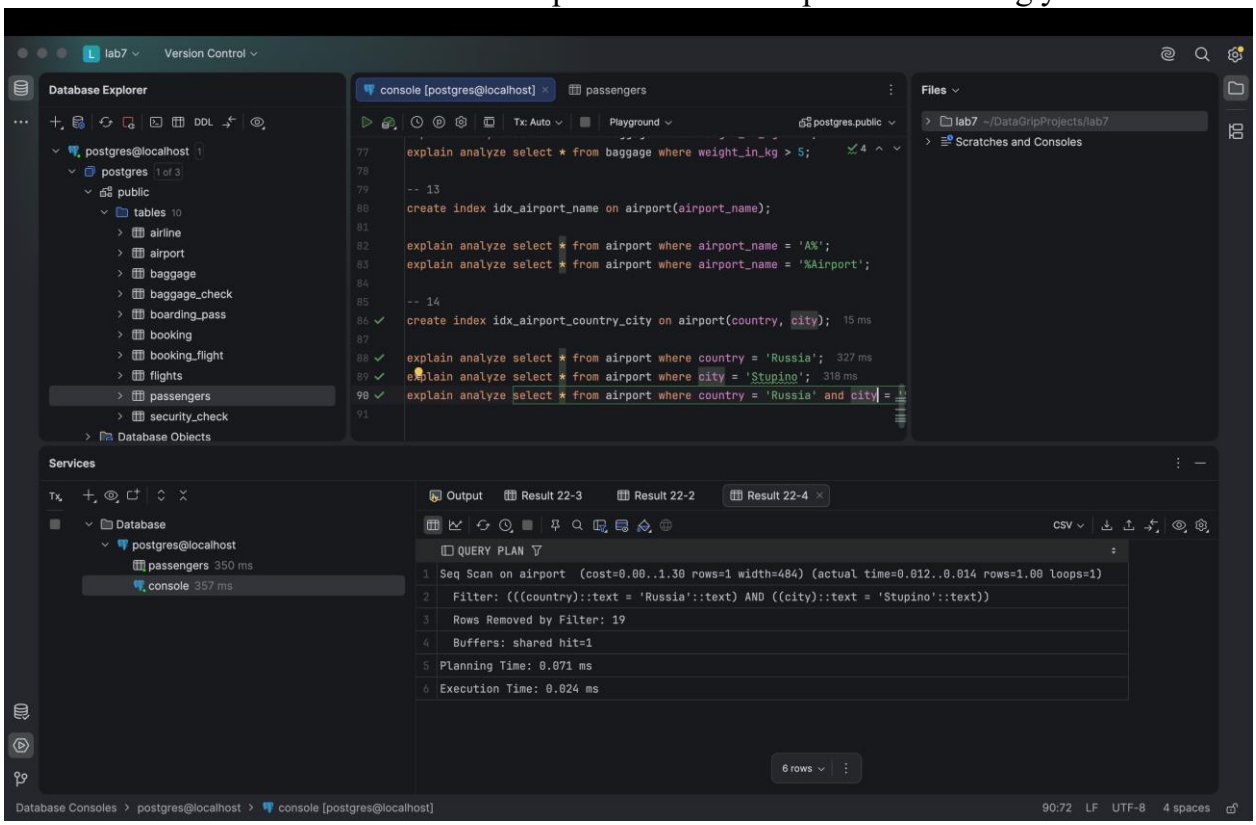
12. Create an index on the weight_in_kg column of the baggage table. Compare the plans of two range queries: weight_in_kg > 49 (a small part of the table) and weight_in_kg > 5 (almost the whole table). Explain the difference in index usage.



13. Create an index on the airport_name column of the airport table. Check with EXPLAIN ANALYZE whether it is used by the queries LIKE 'A%' and LIKE '%Airport'. Explain why the results are different.

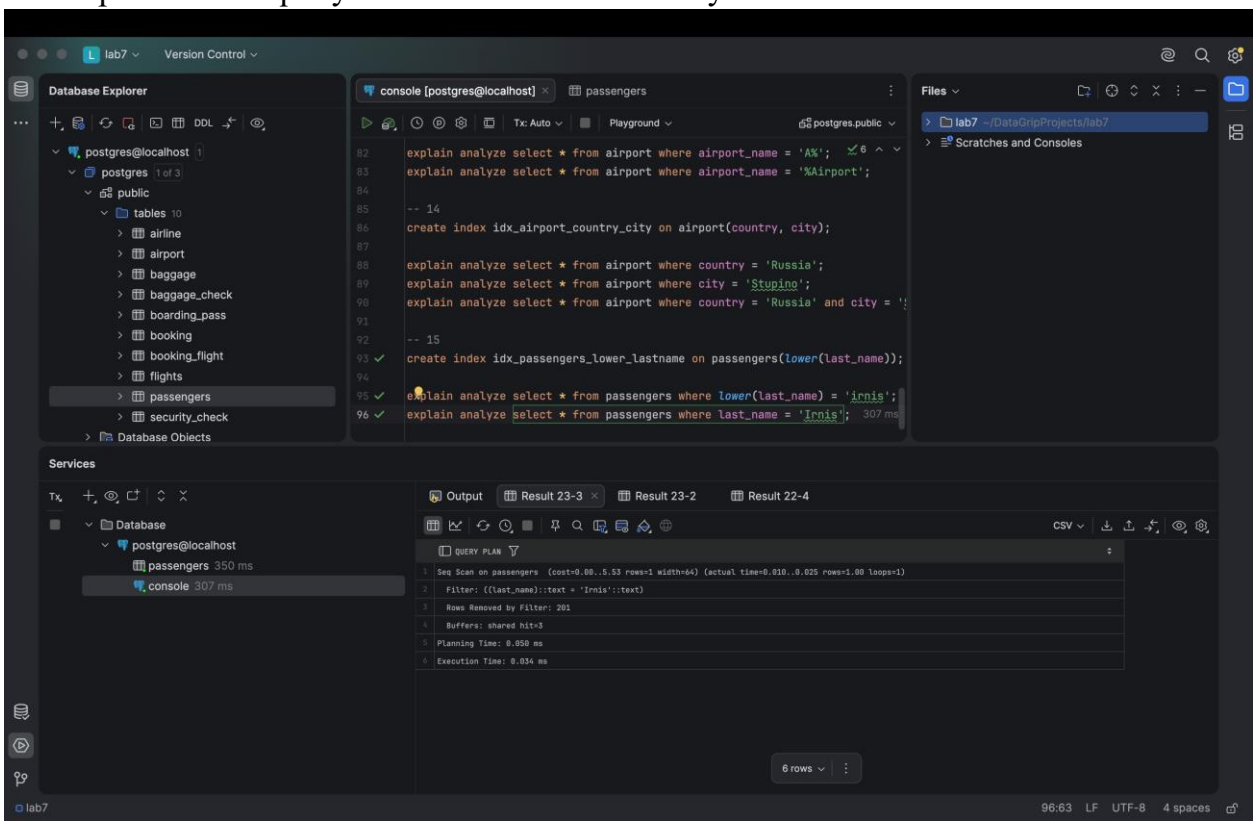


14. Create a composite index on the country and city columns of the airport table. Run three queries: filtering by country only, by city only, and by both columns. Check which of them use the index and explain the leftmost prefix rule using your results.

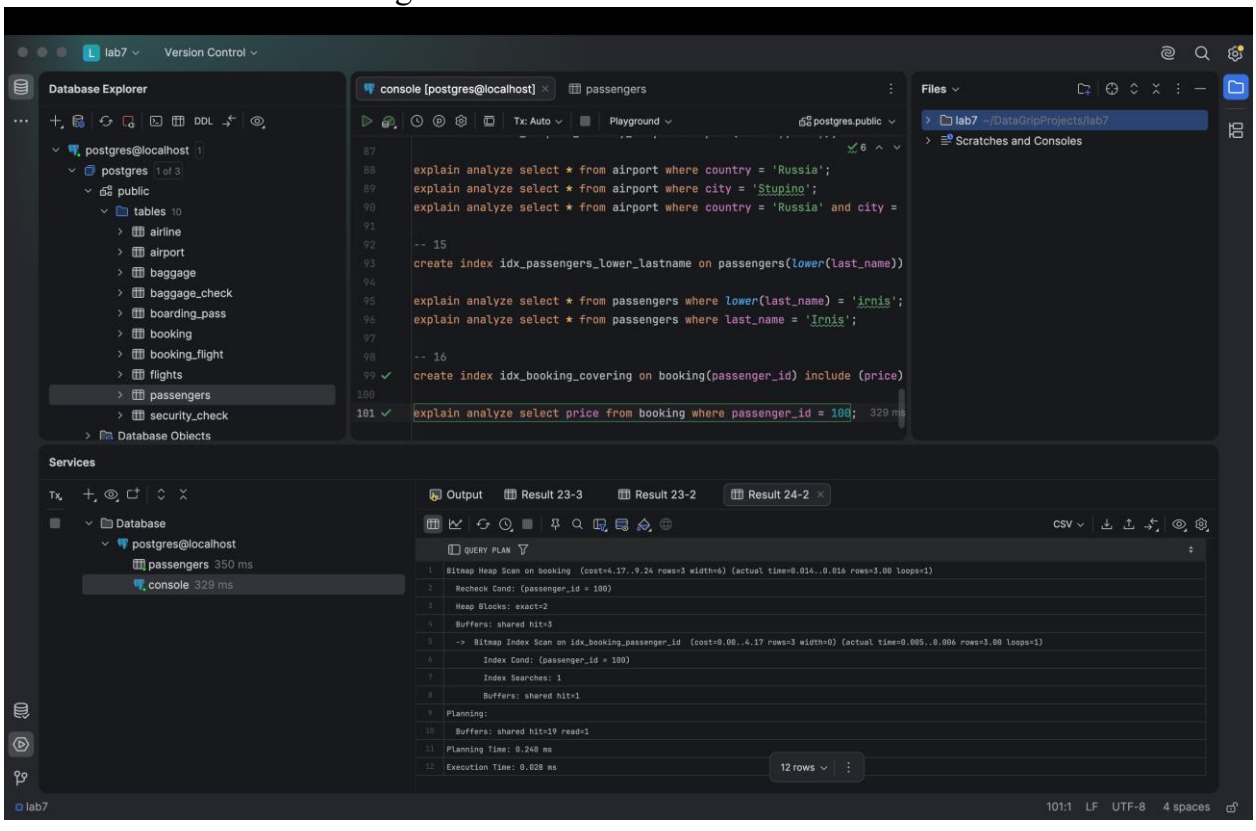


15. Create an expression (functional) index on the lower-case last_name of the Passengers table (LOWER(last_name)). Compare the plans of a query written as WHERE

LOWER(last_name) = 'irnis' and a query written as WHERE last_name = 'Irnis'. Explain which query can use the index and why.



16. Create a covering index on the booking table for the column passenger_id and price. Write a query that returns only price for a given passenger_id and check with EXPLAIN ANALYZE whether an index-only scan (or "Using index") is used. Explain the benefits of a covering index.



17. Try to create a unique index on the airline_code column of the airline table. If an error occurs, explain what caused it and find the problematic rows with a GROUP BY ... HAVING COUNT(*) > 1 query. Then create the unique index in a way that works for the current data, or after fixing the data.

